

Product Requirement Document (PRD): DataPilot AI

1. Executive Summary

DataPilot is a terminal-first, AI-native data science copilot designed to automate the repetitive "grunt work" of the data science lifecycle. By leveraging the OpenAI Agents Runtime and a sandboxed execution environment, DataPilot (via its agent) provides a seamless bridge between raw datasets and actionable insights through automated EDA, feature engineering, and modeling.

2. Problem Statement

Data scientists and analysts spend approximately 80% of their time on data cleaning and preparation. Current tools are either too manual (Jupyter Notebooks) or too "black box" (AutoML). There is a gap for a tool that:

Reduces Context Switching: Moving between documentation, StackOverflow, and the terminal.

Ensures Reproducibility: Many exploratory scripts are "one-offs" that are difficult to replicate.

Accelerates Prototyping: The initial "first pass" at a dataset often takes hours of boilerplate code.

3. Goals & Objectives

Goal 1: Reduce the time from "raw CSV" to "initial model results" by 60%.

Goal 2: Provide a secure environment for AI-generated code execution.

Goal 3: Standardize the exploratory data analysis (EDA) process through a curated modular architecture.

4. Target Audience (User Personas)

The Data Analyst: Needs to evaluate data and suggest best practices for data cleaning and baseline modeling.

The Power Analyst: Wants to automate repetitive visualization and file management tasks via CLI.

The ML Engineer: Needs a quick way to validate dataset quality before building complex pipelines.

5. Functional Requirements

5.1 Agentic Intelligence

Intent Recognition: The agent must interpret natural language commands for data tasks (e.g., "Clean the outliers in the 'price' column").

Reasoning Loops: Utilize the OpenAI Agents Runtime to plan and execute multi-step tasks.

5.2 The Modular Architecture

Filesystem Tools: Ability to create, read, and organize directories within a protected workspace.

Data Tools: Integration with Pandas/NumPy for automated summary statistics, null handling, and correlation matrices.

Automation Tools: One-command generation of standard modeling scripts (Logistic Regression, Random Forest, etc.).

5.3 Execution Environment

Sandboxed Workspace: All operations must be confined to a `./root` directory to prevent accidental data loss or system interference.

State Management: The agent must maintain context across the terminal session.

5.4 User Interface (CLI)

Rich Visuals: Use the Rich library to provide formatted tables, progress bars, and syntax-highlighted code outputs.

Single-Command Launch: A streamlined entry point to initialize the agent.

6. Non-Functional Requirements

Security: AI-generated code must never execute outside the designated sandbox.

Extensibility: The tool architecture must allow for easy addition of new "tools" (e.g., SQL connectors, Cloud Storage).

Performance: Latency for agent reasoning should not exceed 5 seconds for standard planning tasks.

7. User Flow

Initialize: User runs `datapilot start`.

Onboarding: Vanessa greets the user and asks for the dataset path.

Ingestion: DataPilot moves the dataset into `./root` and provides a summary.

Task Execution: User types "Perform EDA and find the top 3 features."

Output: Vanessa uses the Data Toolbelt to generate a report and a .png chart.

8. Success Metrics (KPIs)

Time to Insight (TTI): Average time taken to generate the first model evaluation.

Task Success Rate: Percentage of natural language prompts successfully translated into executed code without errors.

Tool Utilization: Frequency of use for specific modules (Data vs. Filesystem) to prioritize future feature development.

9. Future Roadmap

Phase 2: Multi-agent collaboration (e.g., a "Visualizer" agent working with a "Cleaner" agent).

Phase 3: Support for large-scale data (Dask/PySpark integration).